

NanoPay — Cahier de construction BACKEND

Document de travail séparé pour le développeur backend — conçu pour permettre une intégration directe avec le frontend.

1. Mission du développeur backend

Construire l'API, la base de données, les règles métier, l'authentification, les autorisations, les intégrations de paiement, les webhooks, les QR sécurisés, les notifications et la traçabilité. Le frontend ne doit jamais implémenter les règles financières ou de sécurité comme source de vérité.

Source de référence : l'architecture impose une commande centrale unique reliant Client, Commerçant, Banque et Admin, avec Épargne, Coffre et Crédit comme modes de financement. ■filecite■turn0file0■L5-L6■

2. Modèle de données central

Entités minimales à prévoir

Entité	Responsabilité	Lien principal
User / Client	Identité, accès, statut	Client → Commandes
Merchant	Compte commerçant et validation	Commerçant → Boutique
Bank	Banque partenaire	Banque → Profils/Demandes
Store	Boutique	Merchant → Products
Product	Produit, prix publié, stock	Product → Commandes
Order	Source de vérité centrale	Order → tous les financements
Savings	Épargne progressive	Order → Savings
Vault	Coffre par prélèvement salaire	Order → Vault
CreditProfile	Profil bancaire validé	Client → CreditProfile
CreditRequest	Demande liée à une commande	Order → CreditRequest
Credit	Crédit accepté/financé	CreditRequest → Credit
Payment	Transaction Mobile Money	Order → Payment
PaymentWebhook/Event	Événements externes	Payment → Order
QRToken	Jeton serveur sécurisé	Order → QR
Withdrawal	Retrait en boutique	Order → Withdrawal
MerchantSettlement	Règlement commerçant	Order → Settlement
Notification	Information d'un acteur	Actor → Notification
AuditLog	Traçabilité	Toutes opérations → AuditLog

Une commande doit posséder un identifiant unique (ex. NP-2026-00001245) et constituer la source de vérité commune aux quatre comptes. ■filecite■turn0file0■L221-L224■

3. Statuts et machine d'état

Statuts à implémenter exactement

Objet	Statuts
Client	ACTIF

Objet	Statuts
Commerçant	EN ATTENTE / VALIDÉ / SUSPENDU
Profil crédit	NON CONFIGURÉ / EN ANALYSE / VALIDÉ / REFUSÉ
Demande crédit	BROUILLON / ENVOYÉE / EN ANALYSE / ACCEPTÉE / REFUSÉE
Crédit	ACCEPTÉ / EN ATTENTE DE VIREMENT / FINANÇÉ / TERMINÉ
Épargne	EN COURS / PROLONGATION / ATTEINTE / ÉCHOUÉE / REMBOURSÉE
Coffre	PROGRAMMÉ / PRÉLÈVEMENT EN COURS / OBJECTIF ATTEINT / UTILISÉ / BASCULÉ VERS CRÉDIT
Paieement	INITIÉ / EN ATTENTE / CONFIRMÉ / ÉCHOUÉ
Commande	CRÉÉE / FINANCEMENT EN COURS / FINANÇÉE / PRÊTE À LIVRER / LIVRÉE / TERMINÉE
QR	GÉNÉRÉ / UTILISÉ / EXPIRÉ

Le circuit global retenu est : Client → sélection produit → commande → financement → QR → commerçant → retrait → paiement commerçant → commande terminée. ■filecite■turn0file0■L237-L239■

4. API à fournir au frontend

Authentification & comptes

POST /api/auth/register POST /api/auth/login POST /api/auth/refresh POST /api/auth/logout GET /api/me PATCH /api/me

Marketplace

GET /api/marketplace/cities GET /api/marketplace/communes?cityId=... GET /api/stores?cityId=...&communeId=... GET /api/stores/:storeId GET /api/products/:productId

Commandes

POST /api/orders GET /api/orders/:id GET /api/orders POST /api/orders/:id/select-financing

Épargne

POST /api/orders/:id/savings GET /api/savings/:id GET /api/savings/:id/history POST /api/savings/:id/extension

Crédit

GET /api/credit/profile POST /api/credit/profile POST /api/credit/requests GET /api/credit/requests/:id GET /api/credits/:id

Coffre

POST /api/orders/:id/vault GET /api/vaults/:id POST /api/vaults/:id/use POST /api/vaults/:id/switch-to-credit

QR & retrait

GET /api/orders/:id/qr POST /api/merchant/qr/scan POST /api/merchant/orders/:id/withdrawal-confirm

Commerçant

POST /api/merchant/products PATCH /api/merchant/products/:id DELETE /api/merchant/products/:id GET /api/merchant/orders

Banque

GET /api/bank/credit-profiles PATCH /api/bank/credit-profiles/:id/decision GET /api/bank/credit-requests PATCH /api/bank/credit-requests/:id/decision POST /api/bank/transfers/notice

Admin

GET /api/admin/dashboard GET /api/admin/clients GET /api/admin/merchants GET /api/admin/savings GET /api/admin/credits GET /api/admin/payments GET /api/admin/audit-logs

5. Paiements et webhooks

Le backend reçoit les événements Mobile Money / infrastructure interopérable, identifie la transaction, vérifie son identifiant, l'associe à la commande puis met à jour l'épargne et notifie le client.

Le document de référence exige qu'une même transaction ne soit jamais comptabilisée deux fois et que les paiements soient idempotents. ■filecite■turn0file0■L157-L160■

Contrat webhook

POST /api/webhooks/payment Headers: X-Signature: Body: { providerTransactionId, orderId, amount, currency, status, payerReference, timestamp }

Conserver providerTransactionId avec une contrainte UNIQUE. Vérifier signature, montant, commande, devise et statut avant toute écriture métier.

6. Règles métier obligatoires

- Produit sélectionné avant le mode de financement.
- Prix d'un produit publié définitif ; suppression possible selon les règles.
- Épargne : 0–50 000 FCFA → minimum 1 000 FCFA, max 8 mois ; 50 000–100 000 → minimum 2 000 FCFA, max 18 mois ; >100 000 → minimum 5 000 FCFA, max 36 mois.
- Date de départ de l'épargne = date du premier versement effectivement reçu et validé.
- Échec après durée initiale + prolongation éventuelle de 2 mois : pénalité 15 %, remboursement 85 %.
- Épargne à 100 % : objectif atteint → produit prêt au retrait → QR.
- Crédit : ≤50 000 → max 8 mois ; 50 000–100 000 → max 12 mois ; >100 000 : chaque tranche supplémentaire de 50 000 ajoute 6 mois.
- Coffre réservé aux salariés dont le profil crédit bancaire est déjà validé ; prélèvement automatique sur salaire.
- Paiement commerçant déclenché après retrait confirmé.
- Formules 4 %, 20 %, 15 % et autres paramètres financiers configurables dans le back-office.

7. QR, sécurité et autorisations

Le QR affiché au frontend doit représenter un jeton sécurisé associé côté serveur à la commande. Ne jamais faire confiance aux données métier encodées directement dans un QR.

QR payload côté serveur : opaqueToken GET /api/orders/:id/qr → { qrToken, expiresAt } POST /api/merchant/qr/scan → validation serveur POST /api/merchant/orders/:id/withdrawal-confirm → transition

- RBAC : CLIENT, MERCHANT, BANK, ADMIN.
- Chaque endpoint vérifie propriétaire/acteur + rôle + état de la commande.
- CNIB et documents bancaires stockés de façon privée ; URLs temporaires si exposition nécessaire.
- Journaliser date, heure, identifiant, acteur, montant et statut.
- Rate limiting, validation des entrées, CORS configuré pour le domaine frontend, secrets uniquement dans variables d'environnement.

8. Contrat de connexion Backend ↔ Frontend

Élément	Contrat
Base URL	Variable FRONTEND_API_URL / VITE_API_URL
Format	JSON UTF-8
Auth	Bearer access token + refresh selon implémentation
Erreur	{ code, message, details?, requestId }
Succès	{ data, meta? }
Pagination	page, limit, total, nextPage
Dates	ISO 8601
Montants	FCFA en entier, jamais float
ID	UUID/ID serveur ; orderNumber lisible pour l'UI
Temps réel	WebSocket/SSE si activé, sinon polling contrôlé

Le frontend doit consommer uniquement les contrats d'API documentés ; les calculs critiques et transitions de statut restent côté backend.

9. Livrables backend avant intégration

- README d'installation.
- Schéma DB + migrations + seed de démonstration.
- API documentée OpenAPI/Swagger.
- Collection de tests API.
- Auth + RBAC.
- Marketplace + produits + stock.
- Commandes + trois modes de financement.
- Paiements + webhooks idempotents.
- Crédit + banque.
- QR + retrait.
- Règlement commerçant.
- Notifications.
- Admin + audit.
- .env.example sans secrets.
- Environnement de staging accessible au frontend.